CSD201 PE INSTRUCTIONS

Read the instructions below carefully before you start coding.

Students are ONLY allowed to use:

- Materials available on his/her computer (including JDK, NetBeans...).
- For distance learning: Google Meet, Hangout (for exam monitoring purposes).

Complete the PE by following the steps below:

1. Create a folder to store the given projects, e.g. CSD_given (1). Download the given materials into (1).
2. How to do question 1 (repeat the same for question 2): Open NetBeans, open the given Q1 project, then edit the MyList.java file according to the requirements of the exam. (For Q2, edit the file BSTree.java).
3. Before submitting: **Clean and Build Project** (Shift+F11), then rename the folder **dist** to **run**. (If the folder run already exists, delete it first before renaming).
4. **Submission**: to submit project Q1, first select the Question No (1 or 2), browse and choose the project folder (e.g. 1, Q1, Q1A, or Q2, ...), then click the **Submit** button. Do the same for the remaining questions. **Do not submit** the given project without editing it.
5. **Do not use accented Vietnamese** when writing comments in programs.
6. **Do not add** new **import** statement(s) to the given files.
7. The software tools to be used are: **NetBeans IDE 8.x** and **Java JDK 1.8**. **If at least one of the above requirements is not followed, the exam will get ZERO.**

Troubleshooting:

If the given project (e.g. Q1) runs with an error, run "Clean and Build Project" (Shift+F11). If the error persists, try renaming or copying the project to another name, e.g. from Q1 to Q1X or Q1Y.

Do not pay attention to the real meaning of objects, variables and their values in the questions below.

Question 1: (5 marks)

In this question you should complete several methods in the **MyList.java** file.

The class Laptop with 3 data members: producer, weight and price is given and you do not need to edit it. The MyList class is a linked list of Laptop objects. Complete the following methods:

- **void addLast(String xProducer, int xWeight, int xPrice)** - Note: **xWeight>3** and **xPrice>3**.
- **void f1()** – Do not edit this method. Complete only the addLast(...) method described above. Output in the file **f1.txt** must be the following:

```
(H,6,7) (C,4,5) (X,8,9) (Y,5,8)
```

- **void f2()** – Two Laptop objects v, w are given in this function. Assume the list contains at least 2 elements. Write statements to insert v and w into the list so that v is the **first** and w is the **last** node. Output in the file **f2.txt** must be the following:

```
(D,4,7) (L,10,5) (E,9,7) (Q,7,6)
(V,8,9) (D,4,7) (L,10,5) (E,9,7) (Q,7,6) (W,6,7)
```

- **void f3()** – Delete the node that holds the heaviest Laptop (maximum weight). If more than one node satisfies the requirement, delete the first of them. Output in the file **f3.txt** must be the following:

```
(A,4,4) (D,10,6) (E,9,4) (F,5,8) (Q,10,5) (R,5,7)
(A,4,4) (E,9,4) (F,5,8) (Q,10,5) (R,5,7)
```

- **void f4()** – How many Laptops does the list contain? The output in the f4.txt file should be as follows:

```
(J,4,5) (K,5,6) (L,6,8)
3
```

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*

- **void f5()** – Delete the first two nodes, then sort the elements **ascending by weight** (assuming that there is no duplicated value for weight). The content of the output file **f5.txt** must be the following:

```
(N,6,10) (A,9,13) (W,5,8) (S,7,5) (D,8,9)
(I,4,6) (W,5,8) (S,7,5) (D,8,9)
```

Question 2: (5 marks)

In this question you should complete several methods in the **BSTree.java** file.

The class Cat with 3 data members: place, weight and color is given and you do not need to edit it. The BSTree class is a binary search tree of Cat objects. The variable

The class Cat with 3 data members: place, weight and color is given and you do not need to edit it. The BSTree class is a binary search tree of Cat objects. The variable **weight is the key** of the BSTree, thus it must be unique. Complete the following methods:

- **void insert(String xPlace, int xWeight, int xColor)** - The variable weight is the key of the BSTree, so it must be unique.
- **void f1()** – Add statements in the **insert(...)** method so that any node with xPlace.charAt(0) == 'N' is not inserted to the tree. The output in file f1.txt should be as follows (line 1: Breadth first traversal, line 2: In-order traversal):

```
(X,5,2) (P,1,3) (M,9,5) (R,4,8) (F,7,4) (Q,8,7)
(P,1,3) (R,4,8) (X,5,2) (F,7,4) (Q,8,7) (M,9,5)
```

- **void f2()** – Perform pre-order traversal for the **left branch** of the BST, but write to file f2.txt only the nodes with **color!=4**. **Hint**: Copy the function preOrder(...) to function preOrder2(...) and modify it. Output in the file **f2.txt** must be the following (line 1: Breadth first traversal for whole tree, line 2: pre-order traversal for f2 requirement):

```
(P,7,9) (D,4,3) (G,9,6) (H,2,5) (B,6,4) (X,8,9) (L,1,8)
(D,4,3) (H,2,5) (L,1,8)
```

- **void f3()** – Perform a new traversal method to visit all nodes in the BST with the constraint: visit the nodes in the **descending order of weight**. Output in the file **f3.txt** must be the following:

```
(T,8,-5) (M,7,2) (P,6,3) (R,5,9) (S,4,1) (L,2,5) (Y,1,6)
```

- **void f4()** – Print out **leaf nodes** with **color<7** of the tree. Output in the file **f4.txt** must be the following (line 1: pre-order traversal for whole tree, line 2: leaf nodes for f4 (by pre-order traversal)):

```
(A,6,5) (B,2,7) (P,1,9) (C,3,-8) (F,4,2) (D,5,-3) (E,8,3) (G,7,4) (Q,9,1)
(D,5,-3) (G,7,4) (Q,9,1)
```

- **void f5()** – How many **internal nodes** are there in the BST? Output in the file **f5.txt** must be the following (line 1: in-order traversal, line 2: the amount of internal nodes):

```
(L,1,5) (H,2,6) (Q,3,1) (D,4,3) (B,6,-4) (M,7,9) (X,8,7) (G,9,-6)
4
```